

Architecture & Flow Overview

1. System Architecture Diagram

This diagram illustrates the "Tiered Access" model. Notice how the LLM is isolated behind a "Tool/API Layer" while the User and Backend have direct paths to the data for validation and the "Trace View."

```
graph TD
    subgraph Client_Layer ["Client Layer (Management UI)"]
        User((User))
        UI[React Dashboard - White Mode]
    end

    subgraph Logic_Layer ["System Orchestration (The Brain)"]
        API[FastAPI Backend]
        Agent[Scratch-built Agent Loop]
        Validator[SQL Security Guard / Sanitizer]
    end

    subgraph Data_Layer ["Data Storage (Masked from LLM)"]
        SQLite[(SQLite DB: SQL/CSV Data)]
        Chroma[(ChromaDB: PDF Embeddings)]
        Docs[Raw PDF Storage]
    end

    %% Flow Connections
    User -->|Natural Language| UI
    UI -->|API Request| API
    API --> Agent

    %% Agent Tool Usage
    Agent -->|1. Request SQL| Validator
    Validator -->|Safe Query| SQLite
    SQLite -->|Result Set| Agent

    Agent -->|2. Semantic Search| Chroma
    Chroma -->|Relevant Snippets| Agent

    %% Traceability & Safety
    Agent -->|Contextual Answer + Metadata| API
    API -->|Response + Source IDs| UI
    UI -.->|Direct Fetch Trace| Docs
```

2. User Flow Diagram

This flow emphasizes the "Management-Ready" features: clear feedback, loading states to handle chaotic input, and the drill-down trace view.

```
sequenceDiagram
```

```
    participant U as User (Management)
    participant F as Frontend (React)
    participant B as Backend (FastAPI)
    participant L as LLM (Gemini 2.5)
    participant D as Data (SQL/Chroma)
```

```
    U->>F: Types: "Why is Stellar Run trending?"
```

```
    F->>F: Disable UI / Show Loading Skeleton
```

```
    F->>B: POST /query {text: ...}
```

```
    B->>L: Decide tools for "Stellar Run" & "Trending"
```

```
    L-->>B: Call tool: query_sql(trends_table)
```

```
    B->>D: Execute Validated SQL
```

```
    D-->>B: Return: {view_increase: 40%, city: "Mumbai"}
```

```
    B->>L: Summarize results for Management
```

```
    L-->>B: Answer: "Trending due to 40% growth in Mumbai..."
```

```
    B-->>F: Return Answer + {source: "Sales_DB", row_id: 101}
```

```
    F->>F: Enable UI / Render Chart + Answer
```

```
    U->>F: Clicks "Verify Data" (Trace View)
```

```
    F->>B: GET /document_snippet/101
```

```
    B-->>F: Return Raw Data Row/PDF Snippet
```

```
    F->>U: Displays Highlighted Source
```

3. Updated Thoughts for Log

I've updated the `thoughts.md` file (mentally) with the following reasoning for this specific diagram:

- **Logic Isolation:** The "Validator" isn't just a part of the code; it's a structural barrier between the LLM and the SQLite database.
- **Decoupled Trace:** The "Direct Fetch" (the dotted line in the architecture) is my favorite part. The LLM shouldn't be responsible for fetching the original document; it just provides the "ID" it found. This prevents the model from being overwhelmed by large blobs of text while ensuring the user gets the ground truth.
- **Loading State Control:** Since management users are often impatient, the sequences above clearly define where we "Lock" the UI to prevent the double-request issue you mentioned.